

ARQUITECTURA DE COMPUTADORAS

EXAMEN FINAL 12/07/12

EJERCICIO 1: (40%) Una máquina expendedora de gaseosas está controlada por un circuito secuencial. Dicho circuito recibe como entradas el código de dos bits (tabla 1) de las monedas que va introduciendo el usuario. Como salidas, el circuito debe activar 2 señales, correspondientes a si se debe o no entregar la gaseosa (G), y la cantidad de cambio a reintegrar (C1 y C0), codificadas según las tablas 2 y 3 respectivamente.

El funcionamiento del circuito es el siguiente: El usuario debe introducir monedas hasta alcanzar la cantidad de \$1 (valor de la gaseosa). Una vez que la suma de monedas alcance dicho importe, automáticamente la máquina entrega la gaseosa, retornando además el cambio correspondiente. Por ejemplo, si el usuario ha introducido 1 moneda de 25 y 2 monedas de 50, la máquina deberá entregar la gaseosa y una moneda de 25 (salidas $C1\ C0 = 1\ 0\ 1$)

Tabla 1-entradas
Codificación de la entrada M

M1	M2	Significado
0	0	No hay moneda
0	1	Nueva moneda de \$0.25
1	0	Nueva moneda de \$0.50
1	1	Nueva moneda de \$1

Tabla 2-salida
Codificación de la salida G

G	Significado
0	No entregar
1	Entregar Gaseosa

Tabla 3-salidas
Codificación del cambio C

C1	C0	Significado
0	0	No hay cambio
0	1	Cambio de 0.25
1	0	Cambio de 0.50
1	1	Cambio de 0.75

EJERCICIO 2: (40%) Las direcciones 0x2E y 0x2F contienen números en base 13.

Por ejemplo: (0x2E)=0000xxxx (xxxx representa un número válido en base 13)

Realice un programa en assembler para microcontroladores PIC 16c84 que sume los contenidos de las direcciones 0x2E y 0x2F almacenando el resultado (en base 13) en la dirección 0x2A. Se pide:

- Programa en Assembler.
- Contenidos de W, FSR y PCL al finalizar la ejecución del programa.
- Contenidos de las siguientes posiciones de memoria RAM al finalizar la ejecución del programa, cuyos valores antes de correr el programa son
(0x2E) = 0x09 (0x2F) = 0x0A (0x2A) = 0x13

EJERCICIO 3: (20%) Para una máquina con arquitectura Von Neumann, determinar cuantos bits tendrán cada uno de los buses y registros. Calcular compuertas AND y Entradas si la memoria tiene una arquitectura 2 ½ D para el decodificador, MC, lectura y escritura. Justificar.

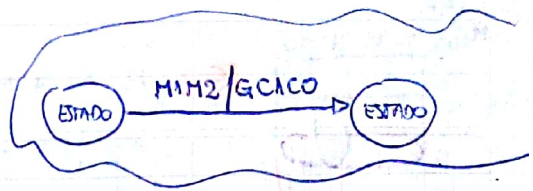
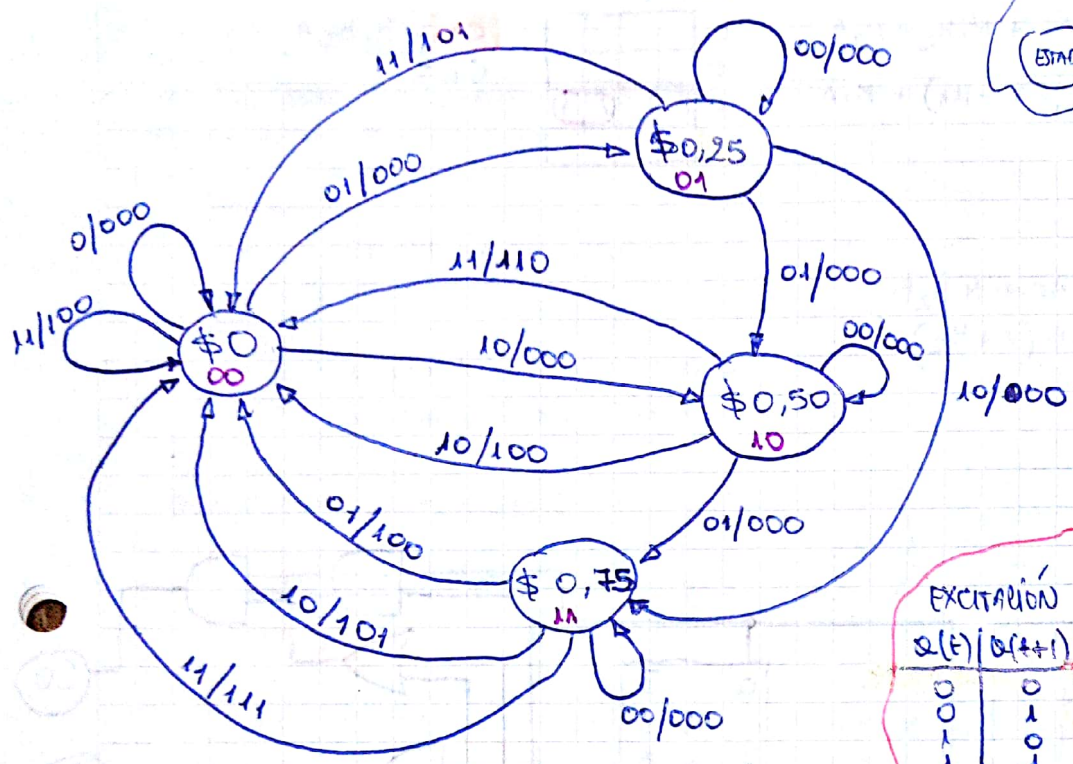
DATOS:

- Memoria de instrucciones 64 Kpalabras de 8K
- Repertorio de instrucciones= 32
- Tipos de direccionamiento: directo, inmediato e indirecto.

Julio 2015

JULIO 2015

FINAL 12/07/12



ENTRADAS $\begin{cases} M1 \\ M2 \end{cases} \rightarrow 2 \text{ Bits}$

ESTADOS = 4 $\rightarrow$ USO 2 BITS

ENT + ESTADOS = 4 BITS
 $2^4 = 16$ FILAS EN LA TABLA

EXCITACIÓN

$Q(t)$	$Q(t+1)$	J	K
0	0	0	X
0	1	X	X
1	0	X	1
1	1	X	0

ENTRADAS		ESTADOS		ESTADOS SIGUIENTES		SALIDAS			FLIP FLOPS			
M_1	M_2	A	B	A'	B'	$G1$	$C1$	$C0$	J_A	K_A	J_B	K_B
0	0	0	0	0	0	0	0	0	0	X	0	X
0	0	0	1	0	1	0	0	0	0	X	X	0
0	0	1	0	1	0	0	0	0	X	0	0	X
0	0	1	1	1	1	0	0	0	X	0	X	0
0	1	0	0	0	1	0	0	0	0	X	1	X
0	1	0	1	1	0	0	0	0	1	X	X	1
0	1	1	0	1	1	0	0	0	X	0	1	X
0	1	1	1	0	0	1	0	0	X	1	X	1
1	0	0	0	1	0	0	0	0	1	X	0	X
1	0	0	1	1	1	0	0	0	1	X	X	0
1	0	1	0	0	0	1	0	0	X	1	0	X
1	0	1	1	0	0	1	0	1	X	1	X	1
1	1	0	0	0	0	1	0	0	0	X	0	X
1	1	0	1	0	0	1	0	1	0	X	X	1
1	1	1	0	0	0	1	1	0	X	1	0	X
1	1	1	1	0	0	1	1	1	X	1	X	1

AB

M_1M_2	00	01	11	10
00			X	X
01		X	X	X
11			X	X
10	1	1	X	X

$J_A = \overline{M_1} M_2 B + M_1 \overline{M_2}$

AB

M_1M_2	00	01	11	10
00			X	X
01	1	X	X	1
11		X	X	
10		X	X	

$J_B = \overline{M_1} M_2$

AB

M_1M_2	00	01	11	10
00	X	X		
01	X	X	1	
11	X	X	1	1
10	X	X	1	1

$K_A = M_1 + M_2 B$

AB

M_1M_2	00	01	11	10
00	X			X
01	X	1	1	X
11	X	1	1	X
10	X		1	X

$K_B = M_2 + M_1 A$

M ₁ M ₂ \ AB	00	01	11	10
00				
01			1	
11	1	1	1	1
10			1	1

$$G = M_2 AB + M_1 M_2 + M_1 A$$

$$M_2 (AB + M_1) + M_1 A$$

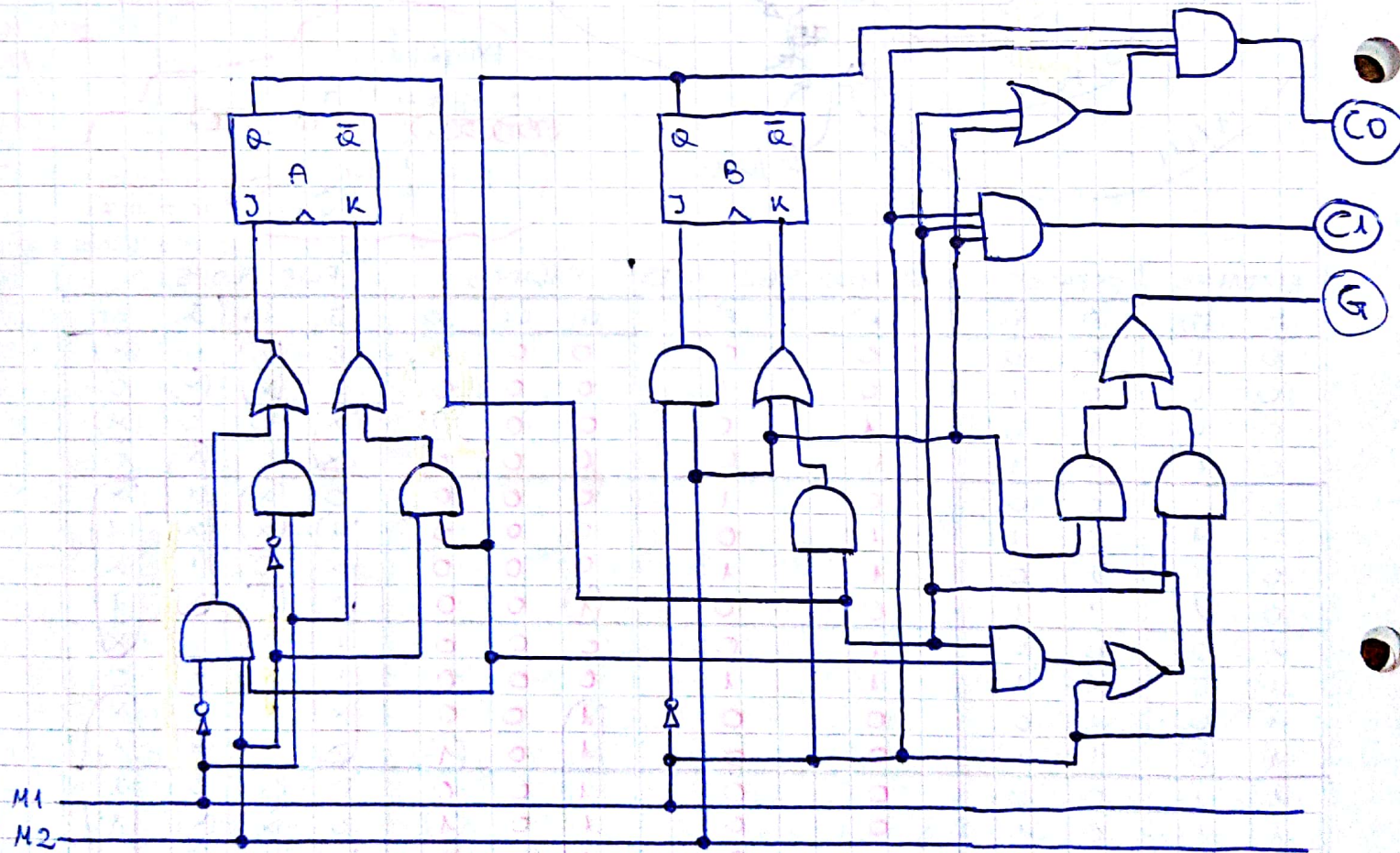
M ₁ M ₂ \ AB	00	01	11	10
00				
01				
11			1	1
10				

$$C_1 = M_1 M_2 A$$

M ₁ M ₂ \ AB	00	01	11	10
00				
01				
11	1	1		
10				1

$$C_0 = M_1 AB + M_1 M_2 B$$

$$M_1 B (A + M_2)$$



2

BASE 13 = DE "0" A "C" (12)

LO QUE HACEMOS ES:

AL ~~10000~~ RESULTADO QUE OBTENGO DE LA SUMA LE SUMO 0X03, PORQUE

$$\begin{array}{r} 0000 \\ \vdots \\ 1100 \end{array} \left. \vphantom{\begin{array}{r} 0000 \\ \vdots \\ 1100 \end{array}} \right\} \text{VÁLIDO EN BASE 13}$$

$$\begin{array}{r} 1101 \\ 1110 \\ 1111 \end{array} \left. \vphantom{\begin{array}{r} 1101 \\ 1110 \\ 1111 \end{array}} \right\} \text{NO VÁLIDO EN BASE 13, PARA CORREGIR DEBO SUMAR 0011(3)}$$

POR EJEMPLO TENGO:

$$\begin{array}{r} 00 \\ + 03 \\ \hline 10 \end{array}$$
 RESULTADO
 NO VÁLIDO EN BASE 13

SI HAY DE (ACARREO INTERMEDIO) QUIERE DECIR QUE EL DÍGITO DE MENOR PESO DEL RESULTADO NO ESTABA EN BASE 13 (COMO EN EL EJEMPLO), OSEA:

$$\begin{array}{r} 000001101 \\ + 000000011 \\ \hline 000001111 \end{array}$$
 (ACARREO DESDE EL BIT 3 O BIEN CUARTO BIT)

DESPUES COMPRUEBO EL DÍGITO DE MAYOR PESO SUMANDO 0X30, Y SI OBTENGO C (ACARREO) ENTONCES QUIERE DECIR QUE TAMPOCO ESTABA EN BASE 13.

POR EJEMPLO, EL RESULTADO ORIGINAL ME DA 0XCD:

$$\begin{array}{r} CD \\ + 03 \\ \hline 100 \end{array}$$
 CORREGIDO

$$\begin{array}{r} 100 \\ + 30 \\ \hline 130 \end{array}$$
 A BASE 13
 130
 CARREO
 NUMERO CORREGIDO

- TENER EN CUENTA QUE, COMO FABA ARRIBA, LA CORRECCIÓN DEL DÍGITO DE LA DERECHA (MENOR PESO), PUEDE HACER QUE EL DE LA IZQUIERDA PASE DE PERTENECER A BASE 13 A NO HABERLO (DEBIDO AL ACARREO)
- EN ASSEMBLER, LOS PROBLEMAS DE CORRECCIÓN DE BASES CASI SIEMPRE SE HACEN ASÍ, USANDO DC, C Y SUMANDO 0X03 Y 0X30, DONDE 3 ES LO QUE DEBO SUMAR PARA CORREGIR EN ESTA BASE

RECORDAR: TANTO EN SUBWF COMO SUBLW SI EL BIT 0 DEL REG. ESTADO (C = CARRY) ES

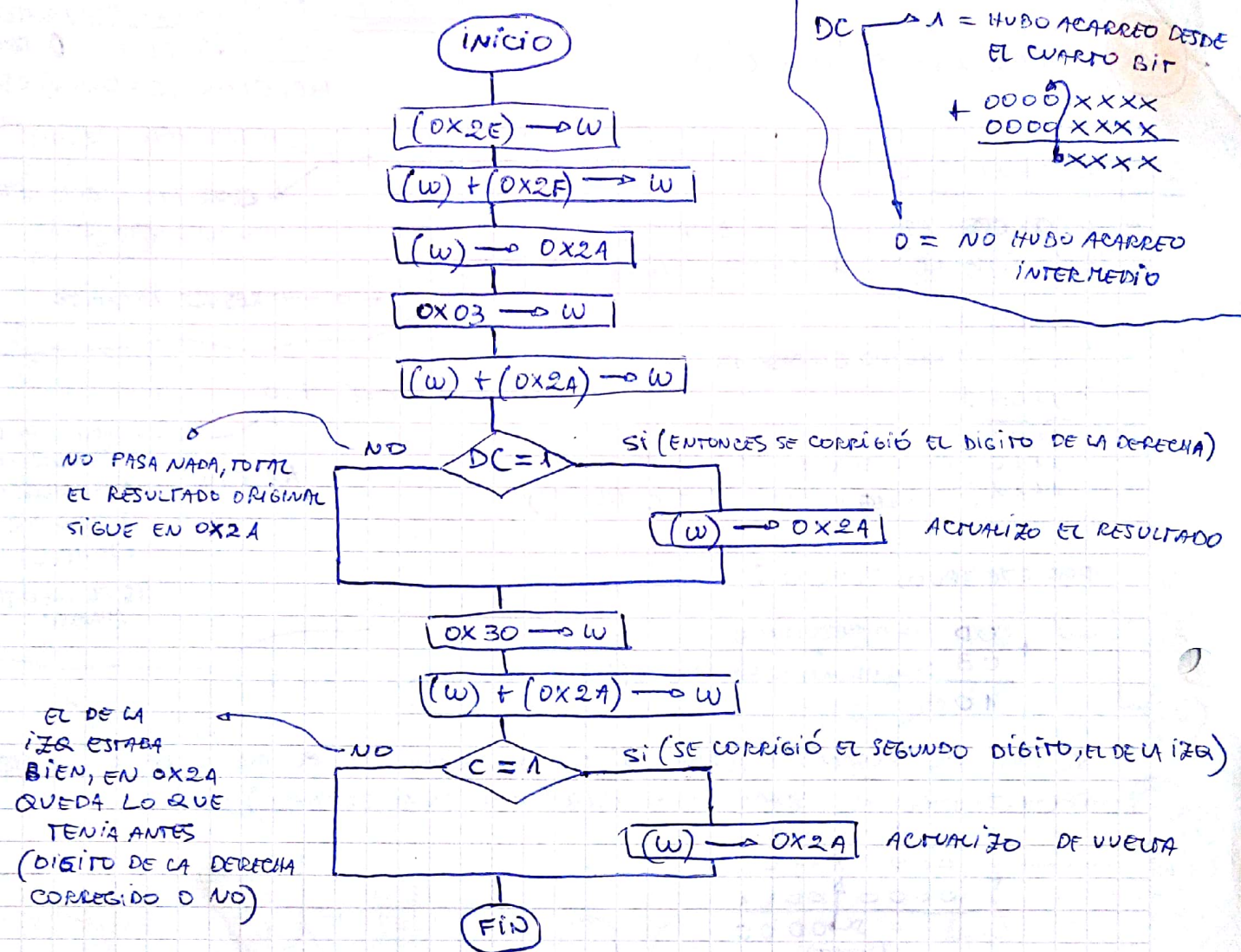
$= 1 \rightarrow$ RESULTADO POSITIVO
 $\rightarrow$ ~~0~~ RESULTADO CERO EN CUYO CASO Z=1

$= 0 \rightarrow$ RESULT NEGATIVO

MOV F, D

0 MUEVE A W

1 MUEVE A F MISMO (PARA COMPROBAR SI ES CERO POR EJEMPLO) ES EL PREDETERMINADO



LIST P = 16CB4

resul	equ 0x2A	org 0x00
0x00	movf 0x2E, 0	
0	addwf 0x2F, 0	
0	movwf resul	
0	movlw 0x03	
	addwf resul, 0	
	btfss STATUS, 1	
	movwf resul	
	movlw 0x30	
	addwf resul, 0	
0x09	btfss STATUS, 0	
0x0A	movwf resul	
	end	

; SALTAR LA SIG. INSTRUCCIÓN SI DC = 1

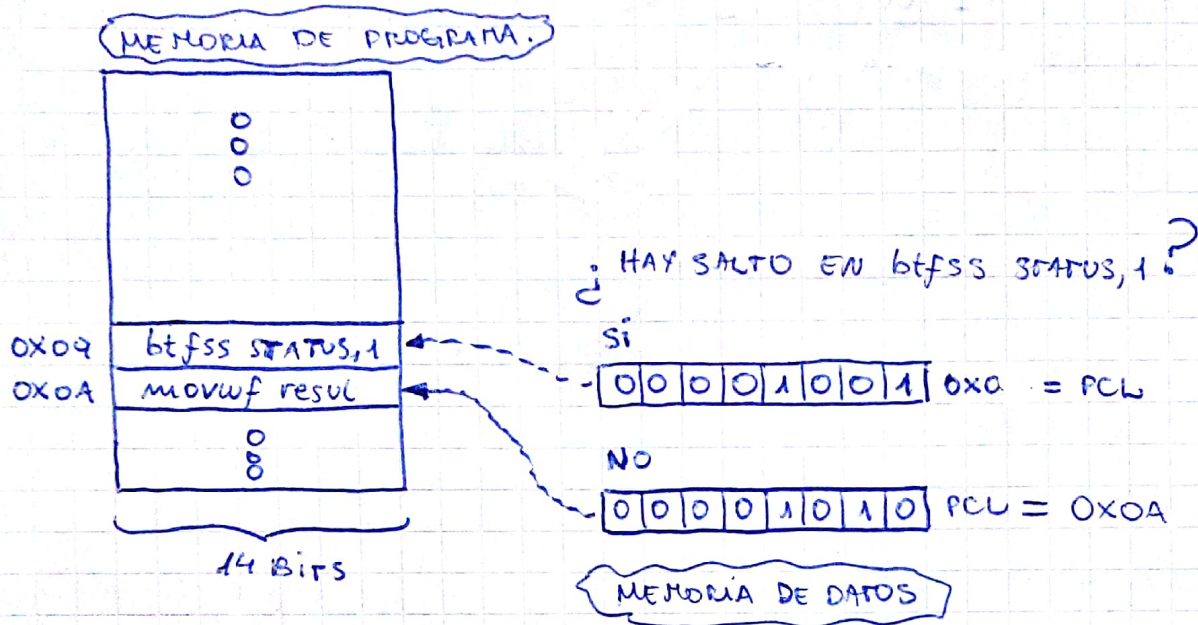
; SALTAR LA SIG. INSTRUCCIÓN SI C = 1

b) W puede quedar con un LITERAL donde los 2 dígitos están corregidos, o que solo el de la izquierda (y si no había que corregir queda con el dígito de la izquierda con +3).

FSR no se usa, por lo que queda en cero.

PCL tiene los 8 BITS DE MENOR PESO DEL PROGRAM COUNTER (PCL = 8 BITS, PC = 13 BITS). Por lo tanto PCL codifica hasta 256 instrucciones. En general

comenzando de 0x00 como dirección de la primera instrucción, PCL terminará conteniendo la dirección de la ~~última~~ última instrucción que se ejecute, en mi caso, si salta en el último BTFSS, ésta es la última que se ejecuta, por lo que (PCL) = 0x09 (que es la dirección en la memoria de programas del último BTFSS. Si no salta (PCL) = 0x0A, que corresponde a MOVWF RESL.



c) (0x2E) = 0x09 (no cambia)
 (0x2F) = 0x0A (" ")

$$\begin{array}{r}
 + 0x09 \\
 0x0A \\
 \hline
 0x13 \rightarrow \text{YA ES CORRECTO}
 \end{array}$$

PRIMERA COMPROBACIÓN:

$$\begin{array}{r}
 + 0x13 \\
 0x03 \\
 \hline
 0x16 \rightarrow \text{NO HAY DC, QUEDA 0x13}
 \end{array}$$

SEGUNDA COMPROBACIÓN:

$$\begin{array}{r}
 + 0x13 \\
 0x30 \\
 \hline
 0x43 \rightarrow \text{NO HAY C, QUEDA 0x13}
 \end{array}$$

(0x2A) = 0x13

EJERCICIO 1: (45%)

En el intervalo de memoria V1 [0x25;0x2F] se tienen cargados valores que representan errores sobre un conjunto de dispositivos. Los bits 0, 1, 2 y 3 indican la gravedad de las fallas que tuvo dicho dispositivo. El significado del tipo de error se interpreta de la siguiente manera:

Bit 0 está puesto a 1: Falla menor

Bit 1 está puesto a 1: Falla crítica recuperada automáticamente

Bit 2 está puesto a 1: Falla crítica recuperada por fuerza bruta

Bit 3 está puesto a 1: Falla Grave, dispositivo bloqueado

Consigna: Escriba un programa en assembler para microcontroladores PIC16c84 que verifique en el intervalo V1 las cargas que registran dispositivos que hayan tenido Fallas Graves y Crítica Recuperada por Fuerza Bruta. Guardar en V2 [0x15;0x1F] las direcciones de V1 que hayan verificado esa condición.

EJERCICIO 2: (45%)

Un sistema recibe secuencialmente datos de 1 bit a través de su entrada X.

Diseñe un circuito que dé salida Z=1 cuando se haya recibido X=1 durante tres o más intervalos de reloj consecutivos.

Un ejemplo del comportamiento que describe el enunciado es el siguiente:

X: 011001011110111110.....

Z: 000000000100011110.....

Realizar:

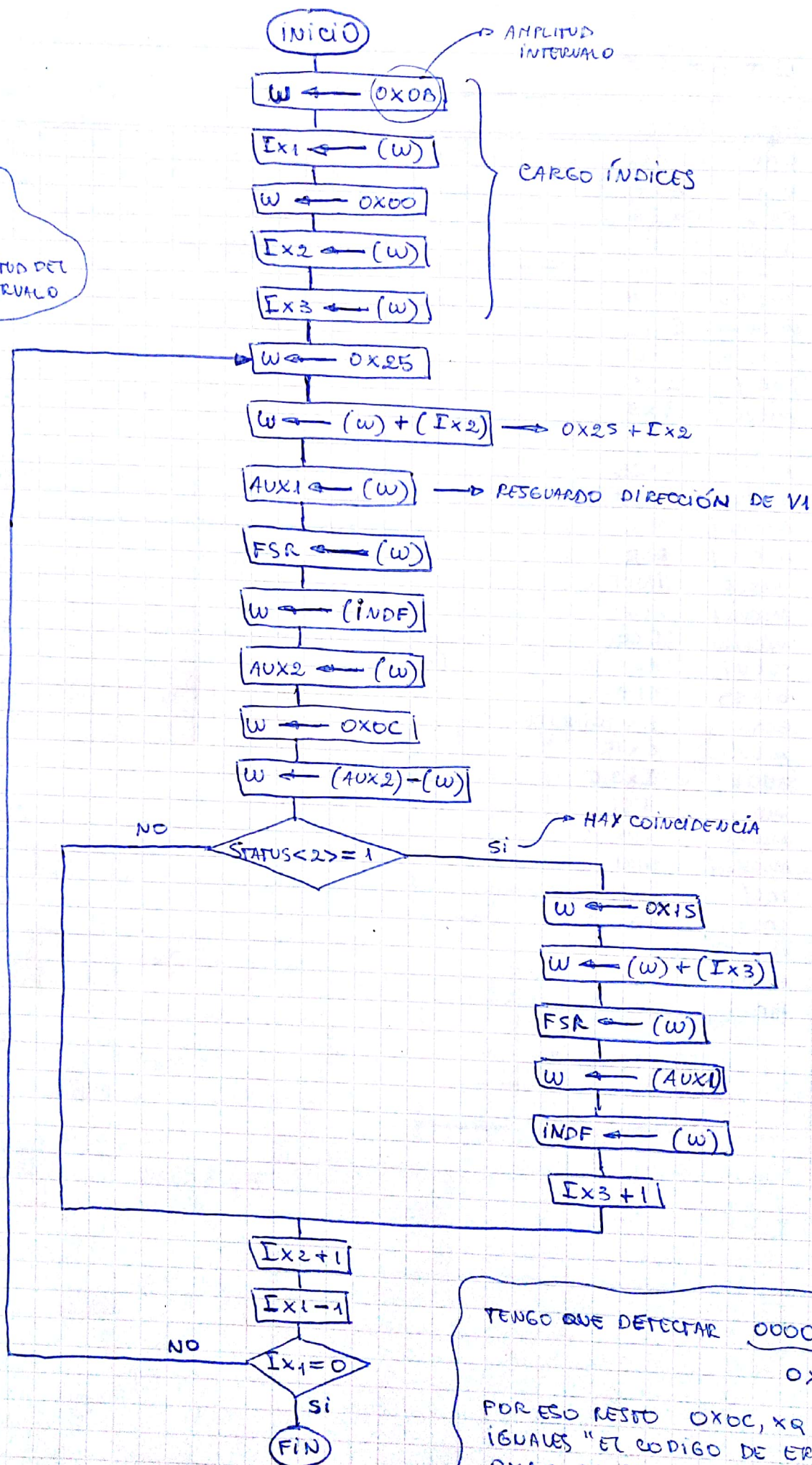
- Diagrama de estados.
- Tabla.
- Simplificaciones
- Circuito con biestables JK.

EJERCICIO 3: (10%)

- Codificar el número -52 en AIKEN, Exceso en 3, BCD zoneado y empaquetado con signo.
- Con el valor en obtenido en el código Aiken, aplicarle Hamming.

1

$$\begin{array}{r} 2F \\ - 25 \\ \hline + 0A \\ 1 \\ \hline 0B \end{array} \rightarrow \text{AMPLITUD DEL INTERVALO}$$



TENGO QUE DETECTAR 0000 11 00
0x0C

POR ESO RESTO 0x0C, XQ SI SON IGUALES "EL CODIGO DE ERROR" Y 0x0C, EL RESULTADO ME DA CERO Y STATUS<2> = 1, PORQUE ES EL BIT DE CERO.

LIST p = 16C84

;-----

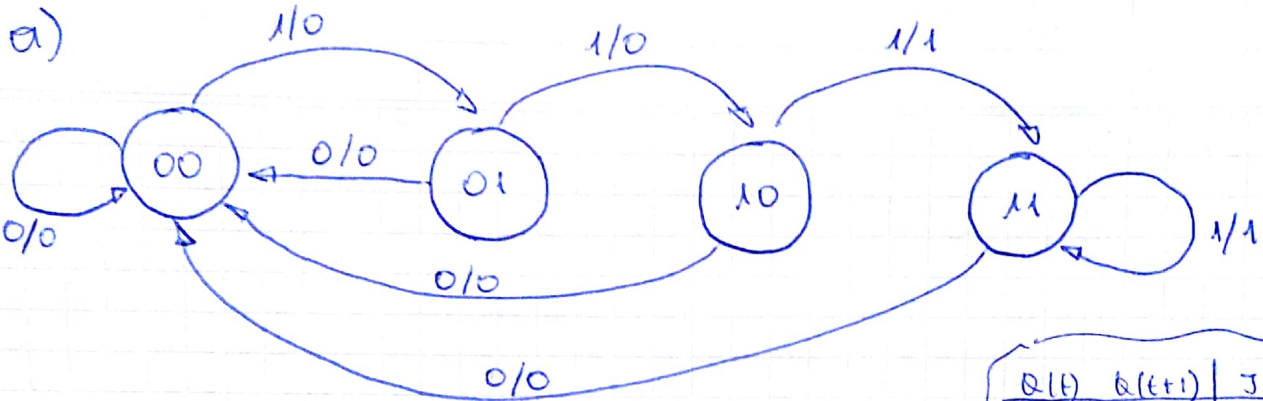
Ix1	equ	0x30
Ix2	equ	0x31
Ix3	equ	0x32
AUX1	equ	0x34
AUX2	equ	0x35

;-----

	movlw	0x0B
	movwf	Ix1
	movlw	0x00
	movwf	Ix2
	movwf	Ix3

BUCLE	movlw	0x25
	addwf	Ix2,0
	movwf	AUX1
	movwf	FSR
	movf	INDF,0
	movwf	AUX2
	movlw	0x0C
	subwf	AUX2,0
	btfss	STATUS,2
	GOTO	NOCOINCIDE
	movlw	0x15
	addwf	Ix3,0
	movwf	FSR
	movf	AUX1,0
	movwf	INDF
	incf	Ix3,1
NOCOINCIDE	incf	Ix2,1
	decfsz	Ix1,1
	GOTO	BUCLE
	fin	

2



Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

PARA 4 ESTADOS $\rightarrow$ 2 BITS

b)

ENTRADA	ESTADOS		ESTADOS SIG		SALIDA	FLIP-FLOPS			
X	A	B	A'	B'	Z	J _A	K _A	J _B	K _B
0	0	0	0	0	0	0	X	0	X
0	0	1	0	0	0	0	X	X	1
0	1	0	0	0	0	X	1	0	X
0	1	1	0	0	0	X	1	X	1
1	0	0	0	1	0	0	X	1	X
1	0	1	1	0	0	1	X	X	1
1	1	0	1	1	1	X	0	1	X
1	1	1	1	1	1	X	0	X	0

c)

AB

X	00	01	11	10
0			X	X
1		X	X	X

$J_A = XB$

AB

X	00	01	11	10
0	X	X	1	1
1	X	X		

$K_A = \bar{X}$

AB

X	00	01	11	10
0		X	X	
1	X	X	X	1

$J_B = X$

AB

X	00	01	11	10
0	X	1	1	X
1	X	1		X

$K_B = \bar{A} + \bar{X}$

AB

X	00	01	11	10
0				
1			1	1

$Z = XA$

d)

